

Running on real IBM Quantum hardware

QF-AgentOS — An honest benchmark and evidence harness for quantum finance

QF-AgentOS contributors

Version 0.1.0 — 2026-07-16

Contents

1. Prerequisites	1
2. Provide the credential — as an environment variable, never in a file	1
3. Opt in, and approve the paid run	1
4. What you get	2
D-Wave (annealing)	2

QF-AgentOS can send the **final optimised QAOA circuit** to real IBM hardware via Qiskit Runtime. The variational parameters are trained on the local statevector simulator (free, fast); only the single best circuit is sampled on the device. This keeps QPU usage — and cost — minimal.

Honesty note. Real-hardware results are noisy and re-verified against the exact constraints like every other backend. A device run that fails (queue, transient error, bad credentials) is recorded and the run completes on the classical baseline — quantum is never faked.

1. Prerequisites

```
pip install 'qf-agentos[ibm]' # qiskit + qiskit-aer + qiskit-ibm-runtime
```

You need an **IBM Quantum Platform** account and an API key (<https://quantum.ibm.com/>). The free **Open Plan** works — QF-AgentOS samples the final circuit in **job execution mode**, which the Open plan allows (it forbids interactive *session* mode).

2. Provide the credential — as an environment variable, never in a file

QF-AgentOS reads the token from the environment as a `SecretStr`; it is never logged, printed, or written into an evidence bundle. **Set it in your own shell — do not commit it or paste it anywhere:**

```
export QF_IBM_TOKEN='your-api-key'
# Optional:
export QF_IBM_CHANNEL='ibm_quantum_platform' # default; or 'ibm_cloud'
export QF_IBM_INSTANCE='<your-instance-CRN>' # if your account needs one
export QF_IBM_BACKEND='ibm_brisbane' # pin a device; else least-busy
```

The legacy `ibm_quantum` channel was retired in `qiskit-ibm-runtime` 0.40+. This platform defaults to `ibm_quantum_platform`; set `QF_IBM_CHANNEL` if your account uses `ibm_cloud`.

Check it is visible to the backend registry:

```
qf-agent backends # 'qaoa_ibm' should show available=true
```

3. Opt in, and approve the paid run

Real hardware is gated: it requires autonomy **L3** and explicit human approval and a sufficient budget. Add to your spec:

```

execution_policy:
  qpu_backend: ibm          # route the final circuit to IBM (default: sim)
  autonomy_level: L3        # RUN_PAID_QPU requires L3
  max_qpu_budget_usd: 5     # must cover the estimated cost (Open Plan is $0)
  max_effective_qubits: 12

```

Then run **with explicit approval**:

```
qf-agent solve examples/collateral-allocation.yaml --approve
```

Without `--approve` the planner still routes to IBM, but the executor refuses the paid run and the pipeline continues on the simulator (you'll see a warning). If credentials are missing, it falls back to the simulator with a recorded reason.

4. What you get

The evidence bundle records the real device name, the QPU time, and the decoded solution re-verified against the full constraints — alongside the classical MILP and the ideal simulator, so the comparison stays honest.

D-Wave (annealing)

The same pattern works for the D-Wave Leap hybrid annealer, which solves the QUBO directly (no gate-model circuit):

```

pip install 'qf-agentos[dwave]'
export QF_DWAVE_TOKEN='your-leap-token' # free Leap tier: 1 min QPU/month
qf-agent backends                       # dwave_hybrid should show available=true

```

```

execution_policy:
  qpu_backend: dwave        # route the QUBO to the annealer
  autonomy_level: L3
  max_qpu_budget_usd: 5

```

```
qf-agent solve examples/collateral-allocation.yaml --approve
```

Same gating (L3 + approval + budget), same honest re-verification; falls back to the simulator with a recorded reason if D-Wave is unavailable.